PostgreSQL

MOHAMMED ABDUL SAMI

Course Plan

- General Database Limits
- Common Database Object Names
- Process and Memory Architecture
- Database Connection
- OBJECTS HIERARCHY
  - Databases
  - Schemas, Schema
  - Search Path
- SQL LANGUAGE
  - Basic Data Types
  - DDL & DML Statements
  - Constraint & Sequences
  - PostgreSQL Operators
- SQL Language
  - SELECT statement all clause
  - TCL Statements (commit and rollback)
  - All Joins types supported by PostgreSQL
  - Table Inheritance
  - Handling NULL values
- ADVANCE SQL LANGUAGE (1 HOUR)
  - Common Table Expressions (CTE)
  - Window Functions
  - Merge Statements
  - RULE System with example
  - Full-text Search (FTS)

Course Plan

- ADVANCE DATA TYPES
- JSON/JSONB with examples
  - Array Data type with examples
  - HStore data types with examples
  - Composite Data type with Examples
- VIEWS
  - Normal Views
  - Materialized Views
- Indexes
  - B-Tree
  - HASH
  - GIN & GiST
  - BRIN

- QUERY TUNING
- Troubleshooting
  - Vacuuming
  - STAT views to identify bottle necks
  - Pg_stat_statement Extension
  - Error Logs
- Statement Processing
- Statistics
- Pg_activity & pg_lock views
- EXPLAIN
- EXPLAIN ANALYZE
- Understanding Query plans
- Restructure SQL

- Cost based optimizations
- Extended Statistics
- Table Partitioning and Performance

Course Plan

- Query Tuning (continued..)
  - CoRelated Sub-Query optimizations
  - SQL Query Best Practices & Guidelines
  - Data Model for Performance
- PROGRAMMING IN
  - PL/pgSQL (basics)
  - Comparing with PL/SQL and SQL
  - Implementing Triggers and Rules for Automation
  - Event-Driven Programming – Utilizing LISTEN/NOTIFY
  - Optimizing SP and func execution
- COPYING AND MOVING DATA
  - pg_dump and pg_dumpall
  - COPY (CSV exports and imports)
  - Using DBeaver and PGAdmin4
- DATABASE UPGRADE & MIGRATION GUIDELINES (2.5 HOURS)
- Migration Strategies: Oracle 2 pg
  - ORa2Pg conversion tool
  - Demo to migrate from Oracle to PostgreSQL using ora2pg tool
  - Using DBeaver and PGAdmin4 tool to exports scripts and schema
- LIQUIBASE
- o Understand what is Change set and Change log
- o Creating Liquibase properties file
- o Creating Liquibase change sets and change logs in SQL, YAML, XML, JSON (1 example
- Liquibase commands

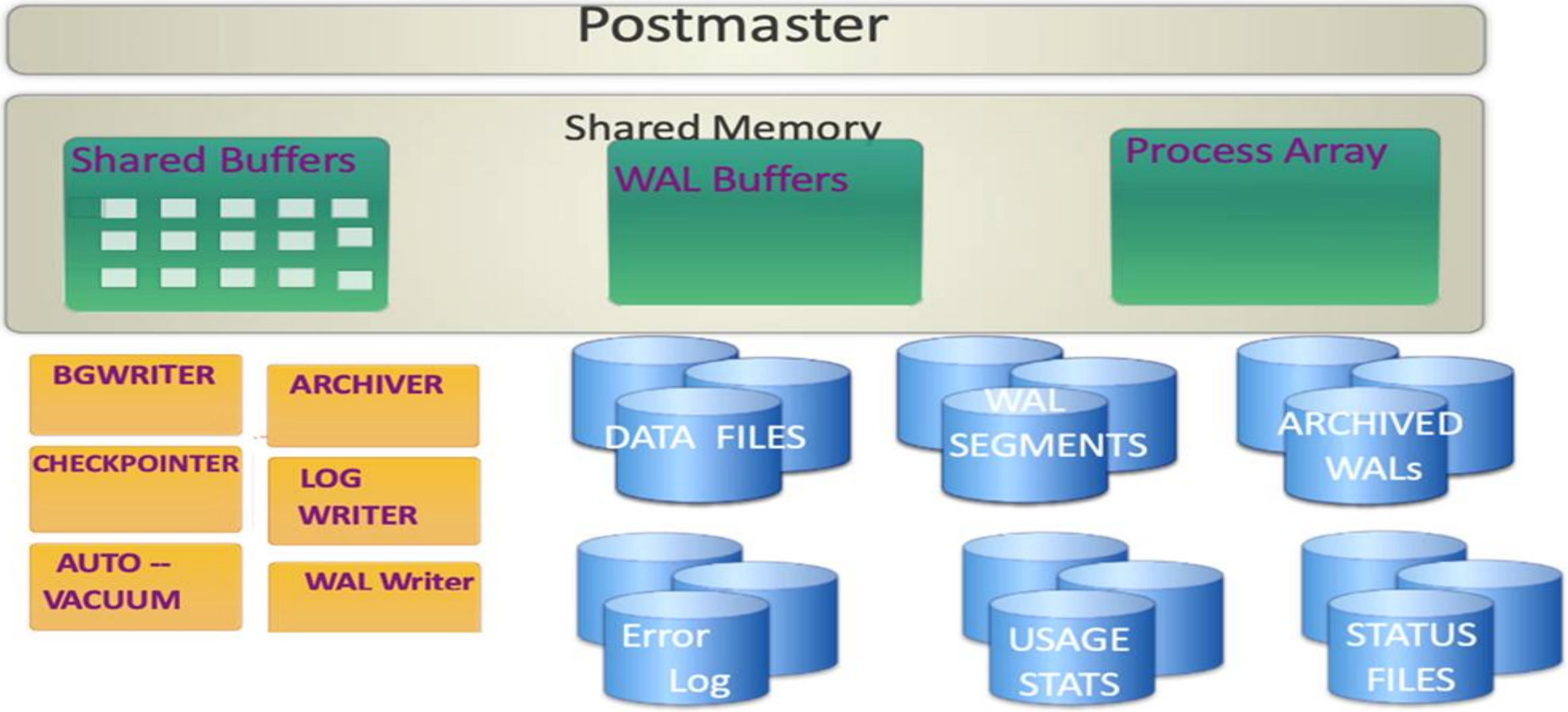
Database Limits

Item	Upper Limit	Comment
database size	unlimited	
number of databases	4294950911	
relations per database	1431650303	
relation size	32 TB	with the default BLCKSZ of 8192 bytes
rows per table	limited by the number of tuples that can fit onto 4,294,967,295 pages	
columns per table	1600	further limited by tuple size fitting on a single page.
columns in a result set	1664	
field size	1 GB	
identifier length	63 bytes	can be increased by recompiling PostgreSQL
indexes per table	unlimited	constrained by maximum relations per database
columns per index	32	can be increased by recompiling PostgreSQL
partition keys	32	can be increased by recompiling PostgreSQL

Database Terms

Industry Term	PostgreSQL Term
Table or Index	Relation
Row	Tuple
Column	Attribute
Data Block	Page (when block is on disk)
Page	Buffer (when block is in memory)

Architecture– Diagram



Architecture– Processes

- Postmaster
  - Postmaster is the master process called Postgres
  - Manages Utility processes, restarts them if they die.
  - Listens on 1-and-only-1 tcp port (default : 5432)
  - Receives client connection request
- Utility Processes
  - Background writer → Writes updated data blocks to disk
  - WAL writer → Flushes write-ahead log to disk
  - Checkpointer process → Automatically performs a checkpoint based on config parameters
  - Logging collector → Routes log messages to syslog, eventlog, or log files
  - Autovacuum launcher → Starts Autovacuum workers as needed
  - Autovacuum workers → Recover free space for reuse
  - Archiver → Archives write-ahead log files

Architecture– Processes

- Postmaster
  - Postmaster is the master process called Postgres
  - Manages Utility processes, restarts them if they die.
  - Listens on 1-and-only-1 tcp port (default : 5432)
  - Receives client connection request
- Utility Processes
  - Background writer → Writes updated data blocks to disk
  - WAL writer → Flushes write-ahead log to disk
  - Checkpointer process → Automatically performs a checkpoint based on config parameters
  - Logging collector → Routes log messages to syslog, eventlog, or log files
  - Autovacuum launcher → Starts Autovacuum workers as needed
  - Autovacuum workers → Recover free space for reuse
  - Archiver → Archives write-ahead log files

Architecture– WAL

- PostgreSQL uses Write-Ahead Logging (WAL) as its approach to transaction logging.
- WAL's central concept is that changes to data files (where tables and indexes reside) must be written only after those changes have been written to WAL files.
- No need to flush data pages to disk on every transaction commit.
- In the event of a crash we will be able to recover the database using the WAL
- This is roll-forward recovery, also known as REDO

Architecture– Rollback

- Dynamic allocation of disk space is used for the storage and processing of records within tables.
- The files that represent the table grow as the table grows.
- It also grows with transactions that are performed against it.
- The data is stored within the file that represents the table.
- When deletes and updates are performed on a table, the file that represents the object will contain the previous data
- This space gets reused, but need to force recovery of used space. (Using Vaccum)

Architecture– DataFile

- File-per-table, file-per-index.
- A table-space is a directory.
- Each database that uses that table-space gets a subdirectory.
- Each relation using that tablespace/database combination gets one or more files, in 1GB chunks.
- Additional files used to hold auxiliary information (free space map, visibility map).
- Each file name is a number (see pg_class.relfilenode)

Architecture– Toast

- Data file page size is 8kb.
- All columns of a rows must fit in the page. Overlapping is not allowed
- Becomes problem for oversized columns.
- To overcome this PostgreSQL introduces The Oversized Attribute Storage Technique (TOAST)
- Handles Oversized column transparently
- Manually variable length columns can be saved in TOAST

```
CREATE TABLE <table name> (name varchar(40), photo bytea STORAGE  EXTENDED COMPRESSED,…);
```

Architecture– Freespace Map & Visibility MAP

- Each Relation has a free space map
  - Stores information about free space available in the relation
  - File named as filenode number plus the suffix _fsm
- Tables also have a visibility map
  - Track which pages are known to have no dead tuples
  - Stored in a fork with the suffix _vm

Architecture– Client Connections

- Connection string in URI format

```
postgresql://[user[:password]@][host][:port][/dbname]
```

- Key-Value Connection String

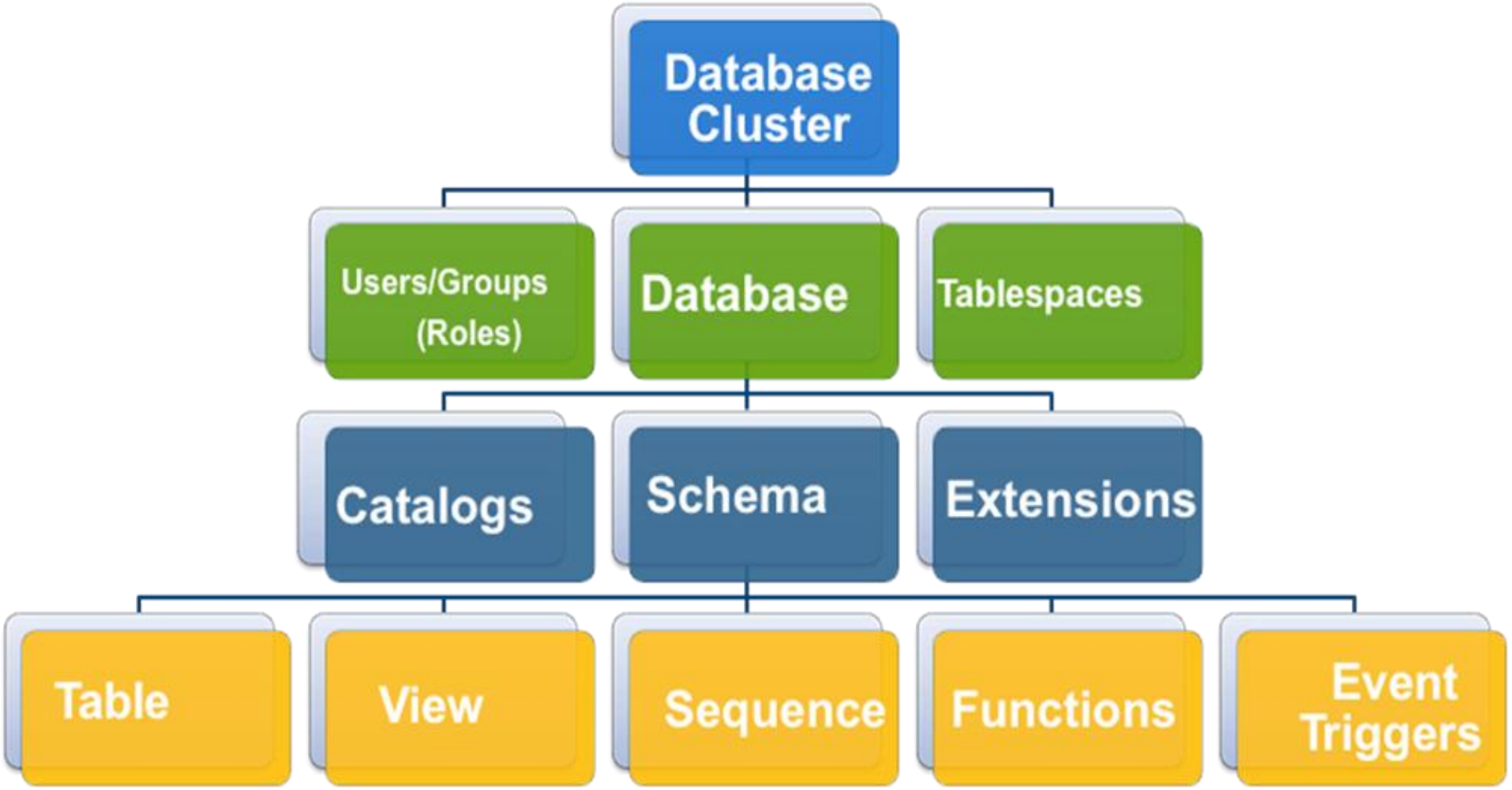
```
'host=<ip/hostname> port=<n> dbname=<dbname>  user=<user> password=<password>'
```

- Java Connection string

```
String jdbcUrl = "jdbc:postgresql://localhost:5432/database_name";
String username = "username";
String password = "password";
```

Note: With every connection request a Database name must be specified.

Architecture– Database hierarchy



Architecture– Database hierarchy

- A running PostgreSQL server can manage multiple databases.
- A database is a named collection of SQL objects.
- It is a collection of schemas and the schemas contain the tables, functions, etc.
- Databases are created with CREATE DATABASE command.
- Databases are destroyed with DROP DATABASE command.

Architecture– Schema

- A database can contains one or more named schemas.
- By Default, all database contain public schema.
- There are several reasons why one might want to use schemas:
- To allow many users to use one database without interfering with each other.
- To organize database objects into logical groups to make them more manageable.
- Third-party applications can be put into separate schemas so they cannot collide with the names of other objects.
- Syntax:

```
CREATE SCHEMA  <schema name>  AUTHORIZATION  <username>;
```

Architecture– search_path

- Object in the database should be referred as <schema_name>.<object_name>.
- This is called as chema qualified object names
- This is a bit tedious.
- Schema qualification for object names can be elimated by using user or session setting called `search_path`
- Syntax:

```
SHOW search_path;
SET search_path TO myschema, public, raj;
ALTER USER <username> IN DATABASE <db name> SET = myschema, public;
```

DataTypes

Datatypes– Integer

Name	Storage Size	Description	Range
smallint	2 bytes	small-range integer	-32768 to +32767
Integer, int	4 bytes	typical choice for integer	-2147483648 to +2147483647
bigint	8 bytes	large-range integer	-9223372036854775808 to +9223372036854775807

Datatypes– Fixed Value Numeric

Name	Storage Size	Description	Range
decimal	variable	user-specified precision, exact	up to 131072 digits before the decimal point; up to 16383 digits after the decimal point
numeric	variable	user-specified precision, exact	up to 131072 digits before the decimal point; up to 16383 digits after the decimal point

Datatypes– Approximate Value Numeric

Name	Storage Size	Description	Range
real	4 bytes	variable-precision, inexact	6 decimal digits precision
double precision	8 bytes	variable-precision, inexact	15 decimal digits precision

Datatypes– Serial

Name	Storage Size	Description	Range
smallserial	2 bytes	small autoincrementing integer	1 to 32767
serial	4 bytes	autoincrementing integer	1 to 2147483647
bigserial	8 bytes	large autoincrementing integer	1 to 9223372036854775807

Datatypes– Monetary

Name	Storage Size	Description	Range
money	8 bytes	currency amount	-92233720368547758.08 to +92233720368547758.07

Datatypes– Monetary

Name	Storage Size	Description	Range
money	8 bytes	currency amount	-92233720368547758.08 to +92233720368547758.07

Datatypes– Character

Name	Description
character varying(n), varchar(n)	variable-length with limit
character(n), char(n), bpchar(n)	fixed-length, blank-padded
bpchar	variable unlimited length, blank-trimmed
text	variable unlimited length

Datatypes– Date

Name	Storage Size	Description	Low Value	High Value
timestamp [(p)] [without time zone]	8 bytes	both date and time (no time zone)	4713 BC	294276 AD
timestamp [(p)] with time zone	8 bytes	both date and time, with time zone	4713 BC	294276 AD
date	4 bytes	date (no time of day)	4713 BC	5874897 AD
time [(p)] [without time zone]	8 bytes	time of day (no date)	00:00:00	24:00:00
time [(p)] with time zone	12 bytes	time of day (no date), with time zone	00:00:00+1559	24:00:00-1559
interval [fields] [(p)]	16 bytes	time interval	-178000000 years	178000000 years

Interval field values:

YEAR, MONTH, DAY, HOUR, MINUTE, SECOND

Datatypes– Binary & Boolean

Name	Storage Size	Description
bytea	1 or 4 bytes plus the actual binary string	variable-length binary string

Name	Storage Size	Description	Valid Values
boolean	1 byte	state of true or false	True, yes,1,on False,no,0,off

Datatypes– Range

- **int4range** — Range of integer, int4multirange — corresponding Multirange
- **int8range** — Range of bigint, int8multirange — corresponding Multirange
- **numrange** — Range of numeric, nummultirange — corresponding Multirange
- **tsrange** — Range of timestamp without time zone, tsmultirange — corresponding Multirange
- **tstzrange** — Range of timestamp with time zone, tstzmultirange — corresponding Multirange
- **daterange** — Range of date, datemultirange — corresponding Multirange

Datatypes– Range.. Operators

Operator	Description	Example	Result
=	equal	int4range(1,5) = '[1,4]'::int4range	t
<>	not equal	numrange(1.1,2.2) <> numrange(1.1,2.3)	t
<	less than	int4range(1,10) < int4range(2,3)	t
>	greater than	int4range(1,10) > int4range(1,5)	t
<=	less than or equal	numrange(1.1,2.2) <= numrange(1.1,2.2)	t
>=	greater than or equal	numrange(1.1,2.2) >= numrange(1.1,2.0)	t
@>	contains range	int4range(2,4) @> int4range(2,3)	t
@>	contains element	'[2011-01-01,2011-03-01)'::tsrange @> '2011-01-10'::timestamp	t
<@	range is contained by	int4range(2,4) <@ int4range(1,7)	t
<@	element is contained by	42 <@ int4range(1,7)	f
&&	overlap (have points in common)	int8range(3,7) && int8range(4,12)	t
<<	strictly left of	int8range(1,10) << int8range(100,110)	t
>>	strictly right of	int8range(50,60) >> int8range(20,30)	t
&<	does not extend to the right of	int8range(1,20) &< int8range(18,20)	t
&>	does not extend to the left of	int8range(7,20) &> int8range(5,10)	t
-|-	is adjacent to	numrange(1.1,2.2) -|- numrange(2.2,3.3)	t
+	union	numrange(5,15) + numrange(10,20)	[5,20)
*	intersection	int8range(5,15) * int8range(10,20)	[10,15)
-	difference	int8range(5,15) - int8range(10,20)	[5,10)

Datatypes– Range.. Functions

Function	Return Type	Description	Example	Result
lower(anyrange)	range's element type	lower bound of range	lower(numrange(1.1,2.2))	1.1
upper(anyrange)	range's element type	upper bound of range	upper(numrange(1.1,2.2))	2.2
isempty(anyrange)	boolean	is the range empty?	isempty(numrange(1.1,2.2))	false
lower_inc(anyrange)	boolean	is the lower bound inclusive?	lower_inc(numrange(1.1,2.2))	true
upper_inc(anyrange)	boolean	is the upper bound inclusive?	upper_inc(numrange(1.1,2.2))	false
lower_inf(anyrange)	boolean	is the lower bound infinite?	lower_inf('(,)'::daterange)	true
upper_inf(anyrange)	boolean	is the upper bound infinite?	upper_inf('(,)'::daterange)	true

Datatypes– Range.. Functions

Function	Return Type	Description	Example	Result
lower(anyrange)	range's element type	lower bound of range	lower(numrange(1.1,2.2))	1.1
upper(anyrange)	range's element type	upper bound of range	upper(numrange(1.1,2.2))	2.2
isempty(anyrange)	boolean	is the range empty?	isempty(numrange(1.1,2.2))	false
lower_inc(anyrange)	boolean	is the lower bound inclusive?	lower_inc(numrange(1.1,2.2))	true
upper_inc(anyrange)	boolean	is the upper bound inclusive?	upper_inc(numrange(1.1,2.2))	false
lower_inf(anyrange)	boolean	is the lower bound infinite?	lower_inf('(,)'::daterange)	true
upper_inf(anyrange)	boolean	is the upper bound infinite?	upper_inf('(,)'::daterange)	true

Datatypes– Range.. Example

Examples:

```
CREATE TABLE tennis_court (court_id int, reserved tsrange);
```

Insert data:

```
INSERT INTO tennis_court VALUES (10, '[2024-11-25 8:00, 2024-11-25 10:00]');
INSERT INTO tennis_court VALUES (11, '[2024-11-25 8:00, 2024-11-25 9:30]');
```

Check if the court 10 is booked between 9:30 to 11:30 on 2024-11-25

```
select * from tennis_court where reserved && tsrange('2024-11-25 09:30', '2024-11-25
11:30') and court_id=10;
 court_id |                    reserved
----------+-----------------------------------------------
       10 | ["2024-11-25 08:00:00","2024-11-25 10:00:00"]
```

Datatypes– Range.. Constraint

EXCLUSION Constraint:

- Can be used with Range Data type and GIS Data Type
- Restricts overlapping entries in the column

Example:

```
CREATE TABLE reservations (
room_id INTEGER,
booking_duration    tsrange,
EXCLUDE USING GIST (room_id  WITH =,   booking_duration WITH &&)
);
```

Operators

Operators– Comparison

Operator	Description	Usage
<	Less than?	1 < 2
<=	Less than or equal to?	1 <= 2
<>	Not equal?	1 <> 2
=	Equal?	1 = 1
>	Greater than?	2 > 1
>=	Greater than or equal to?	2 >= 1
||	Concatenate strings	Postgre' || 'SQL'
~~	LIKE	scrappy,marc,hermit' ~~ '%scrappy%'
!~~	NOT LIKE	bruce' !~~ '%al%'
~	Match (regex), case sensitive	thomas' ~ '*.thomas*.'

Operators– Comparison

Operator	Description	Usage
!~	Does not match (regex), case sensitive	thomas' !~ '*.Thomas*.'
!~*	Does not match (regex), case insensitive	thomas' !~ '*.vadim*.'
IS NULL	Test whether value is NULL	
IS NOT NULL	Negates NULL test	
IS TRUE	Test whether value is true	
IS DISTINCT	Test whether values are distinct	x IS DISTINCT y
LIKE	Pattern Matching	X LIKE '%s%'
NOT LIKE	Negates pattern Matching	
IN		X in (v1, v2,..vn)
NOT IN		
BETWEEN	Check if the given value is between given tange	
NOT BETWEEN	Negates the BETWEEN check	

Operators– Arithmetic

Operator	Description	Example	Result
+	addition	2 + 3	5
-	subtraction	2 - 3	-1
*	multiplication	2 * 3	6
/	division (integer division truncates the result)	4 / 2	2
%	modulo (remainder)	5 % 4	1
^	exponentiation (associates left to right)	2.0 ^ 3.0	8
|/	square root	|/ 25.0	5
||/	cube root	||/ 27.0	3
@	absolute value	@ -5.0	5

Operators– Arithmetic

Operator	Description	Example	Result
+	addition	2 + 3	5
-	subtraction	2 - 3	-1
*	multiplication	2 * 3	6
/	division (integer division truncates the result)	4 / 2	2
%	modulo (remainder)	5 % 4	1
^	exponentiation (associates left to right)	2.0 ^ 3.0	8
|/	square root	|/ 25.0	5
||/	cube root	||/ 27.0	3
@	absolute value	@ -5.0	5

DDL Statements

DDL Statements

- Create the Objects
- Modify the Objects
- Drop Objects
- Truncate Table

DDL Statements.. Create Table Statement

```
CREATE [TEMPORARY][UNLOGGED] TABLE table_name
( [column_name data_type [ column_constraint], ..... )
[ INHERITS ( parent_table) ]
[ TABLESPACE tablespace_name ]
[ USING INDEX TABLESPACE tablespace_name ]
```

DDL Statements.. Constraints

- Constraints can be defined at the column level or table level
- Constraints can be added to an existing table using the ALTER TABLE statement
- Constraints can be declared DEFERRABLE or NOT DEFERRABLE
- Constraints prevent the deletion of a table if there are dependencies
    - NOT NULL
    - CHECK
    - UNIQUE
    - PRIMARY KEY
    - FOREIGN KEY
    - EXCLUDE (Requires EXTENSION btree_gist)

Example of EXCLUDE constraint:

```
CREATE TABLE room_reservation (
    id SERIAL PRIMARY KEY,
    room_id INTEGER NOT NULL,
    reserved_during TSRANGE NOT NULL,
    EXCLUDE USING GIST (
        room_id WITH =,
        reserved_during WITH &&
    )
);
```

DDL Statements.. Constraints

- PostgreSQL provides SEQUENCE object, SERIAL Datatype and IDENTITY column
- SERIAL Datatype generates sequence of integers and used with Primary Key columns
- IDENTITY columns are little advanced than SERIAL datatype.
    - IDENTITY columns are part of SQL16 standard and compatible with other databases
    - Recommended to prefer IDENTITY columns over SERIAL Datatype
    - They are more flexible than SERIAL datatype

```
CREATE TABLE orders (
    id BIGINT GENERATED ALWAYS AS IDENTITY ( START WITH 100 INCREMENT BY 5 MAXVALUE 999999 ),
    amount NUMERIC
);
```

- SEQUENCE gives better performance on a heavy INSERT environment as it can cache more than 1 NEXTVAL

DDL Statements.. Create Sequence Syntax

```
CREATE SEQUENCE [ IF NOT EXISTS ] sequence_name
    [ AS { SMALLINT | INT | BIGINT } ]
    [ INCREMENT [ BY ] increment ]
    [ MINVALUE minvalue | NO MINVALUE ]
    [ MAXVALUE maxvalue | NO MAXVALUE ]
    [ START [ WITH ] n ]
    [ CACHE n ]
    [ [ NO ] CYCLE ]
    [ OWNED BY { table_name.column_name | NONE } ]
```

DDL Statements.. Rule

- Rules are some where between Constraint and Trigger
- Like Triggers, they also fires on DML events
- Can be created for Tables & Views
- Like Triggers, they also sees NEW and OLD data structures

DDL Statements.. Rule

Syntax:

```
CREATE [OR REPLACE] RULE rule_name AS
    ON event
    TO table_name
    [WHERE condition]
    DO [ALSO | INSTEAD] action;
```

rule_name: The name of the rule.

event: One of SELECT, INSERT, UPDATE, or DELETE.

table_name: The table or view on which the rule applies.

condition: Optional condition that must be satisfied for the rule to apply.

action: One or more SQL commands to be executed when the rule is triggered. Use ALSO to execute the original action as well, or INSTEAD to replace it.

DDL Statements.. Rule

Example:

```
CREATE TABLE logs_2024 (
    id serial PRIMARY KEY,
    message text,
    created_at timestamp DEFAULT NOW()
);

CREATE VIEW logs AS SELECT * FROM logs_2024;

 CREATE RULE logs_insert AS
     ON INSERT TO logs
     DO INSTEAD
     INSERT INTO logs_2024 (message)
     VALUES (NEW.message);
```

DDL Statements.. ALTER STMNT

- Add a new column
- Modify an existing column
- Add a constraint
- Drop a column
- Change schema
- Rename a column, constraint or table
- Enable and disable triggers and rules on a table Change ownership
- Enable or disable logging for a table

DDL Statements.. ALTER STMNT

```
ALTER TABLE table_name ADD COLUMN column_name datatype column_constraint;
ALTER TABLE table_name  DROP COLUMN column_name;
ALTER TABLE table_name  RENAME COLUMN column_name  TO new_column_name;
ALTER TABLE table_name ALTER COLUMN column_name [SET DEFAULT value | DROP DEFAULT];
ALTER TABLE table_name ALTER COLUMN column_name [SET NOT NULL| DROP NOT NULL];
ALTER TABLE table_name ADD CHECK expression;
ALTER TABLE table_name ADD CONSTRAINT constraint_name constraint_definition;
ALTER TABLE table_name RENAME TO new_table_name;
```

DML Statements

DML Statements.. DML STMNT

- INSERT
- UPDATE
- DELETE

All above statements can return values using option clause RETURNING

DML Statements.. SELECT Stmnt

The SELECT statement can be used to retrieve (select) data from a table or an expression. It is one of the most complex statement in SQL language with a lot of clauses to perform data retrieval and transformation.

- Select distinct rows using DISTINCT operator.
- Sort rows using ORDER BY clause.
- Filter rows using WHERE clause.
- Select a subset of rows from a table using LIMIT or FETCH clause.
- Group rows into groups using GROUP BY clause.
- Filter groups using HAVING clause.

Syntax:

```
SELECT  [DISTINCT]   [columns selection]  [FROM  <table>]   [WHERE  <expression>]
[GROUP BY <Expression>  [HAVING <Expression>]]  [ORDER BY  <sort expression> ASC |
DESC]  [LIMIT row_count OFFSET rows_to_skip]
```

DML Statements.. SELECT Stmnt Tips

- Avoid SELECT * — Only Fetch Needed Columns
- Use Indexes (WHERE, GROUP BY,JOIN & ORDER BY)
- Leverage Index-Only Scans
- Limit Results When Possible.
- OFFSET is slow. Use WHERE, ORDER BY and LIMIT instead
- Keep Optimizer Statistics Updated
- Tune settings like WORK_MEM SHARED_BUFFER and EFFECTIVE_CACHE_SIZE
- Avoid Functions/Expression/typecasting on Indexed Columns
- Use Covering Indexes
- Use Materialized Views
- Use External connection pooling and query caching layer

DML Statements.. Column Aliasing

- Column aliases can be used in the SELECT list of a SQL query in PostgreSQL.
- Like all objects, aliases will be in lowercase by default. If mixed-case letters or special symbols, or spaces are required, quotes must be used.
- Column aliases can be used for derived columns.
- Column aliases can be used with GROUP BY and ORDER BY clauses.
- We cannot use a column alias with WHERE and HAVING clauses.

DML Statements.. Table Aliasing

- Table aliases can be used in SELECT lists and in the FROM clause to show the complete record or selective columns from a table.
- Table aliases can be used in WHERE, GROUP BY, HAVING, and ORDER BY clauses.
- When we need data from multiple tables, we need to join those tables by qualifying the columns using table name/table alias.
- The aliases are mandatory for inline queries (queries embedded within another statement) as a data source to qualify the columns in a select list.

DML Statements.. Quoting

- Single quotes and dollar quotes are used to specify non-numeric values
- Double quotes are used for names of database objects which either clash with keywords, contain mixed case letters, or contain characters other than a-z, 0-9 or underscore
- Double Dollar ($$) quotes can be used instead of single quote

DML Statements.. Quoting

- Single quotes and dollar quotes are used to specify non-numeric values
- Double quotes are used for names of database objects which either clash with keywords, contain mixed case letters, or contain characters other than a-z, 0-9 or underscore
- Double Dollar ($$) quotes can be used instead of single quote
- Double dolloar ($$) quotes can be labeled ex: `$foo$Hello World$foo$`

DML Statements.. Typecasting

- Converts value from one datatype to another
- Can be done in 3 different ways
  - PostgreSQL specific
    - :: operator (eg: ‘2025-04-10 09:53:40’::timestamp)
    - <datatype> <values/variable/column> (eg: timestamp ‘2025-04-10 09:53:40’)
  - SQL Standard (pottable to other databases)
    - Cast function (eg: cast(‘2025-04-10 09:53:40’ as timestamp)
- Performance tips
  - Prefer :: operator over CAST function as it is a bit faster
  - Never cast integer to decimal/numeric as it is done automatically
  - Try to type cast at application level or if possible at frontend elevel to reduce load on db
  - All type of typecasting ignore indexes

DDL Statements.. Date Arithmetic

Date Additions

```
select now() + interval '13 days'
select now() + interval '1 year'
select now() + interval '1 month'
select date '2021-02-15' + interval '1 month'
```

Interval can be in DAYS, MONTH, YEAR, HOURS, MINUTES and SECONDS

Date Subtraction

```
select date '2021-02-15' - date '2021-03-15' ;
select date '2021-02-15' + interval '30 days';
select ename, now()-hiredate from emp;
select ename, (now()-hiredate) from emp;
select ename, age(now(),hiredate) from emp;
```

DML Statements.. Datatype Formatting functions

- `to_char()`
- `to_date()`
- `to_number()`
- `to_timestamp()`

DML Statements.. Datatype Formatting functions

Pattern	Description
HH	hour of day (01–12)
HH12	hour of day (01–12)
HH24	hour of day (00–23)
MI	minute (00–59)
SS	second (00–59)
MS	millisecond (000–999)
AM, am, PM or pm	meridiem indicator (without periods)
YYYY	year (4 or more digits)
BC, bc, AD or ad	era indicator (without periods)
MONTH	full upper case month name (blank-padded to 9 chars)

DML Statements.. Datatype Formatting functions

Pattern	Description
MON	abbreviated upper case month name (3 chars in English, localized lengths vary)
MM	month number (01–12)
DAY	full upper case day name (blank-padded to 9 chars)
DY	abbreviated upper case day name (3 chars in English, localized lengths vary)
DD	day of month (01–31)
TZ	upper case time-zone abbreviation (only supported in to_char)

DML.. Datatype Formatting functions

Pattern	Description
9	digit position (can be dropped if insignificant)
0	digit position (will not be dropped, even if insignificant)
. (period)	decimal point
, (comma)	group (thousands) separator
PR	negative value in angle brackets
S	sign anchored to number (uses locale)
L	currency symbol (uses locale)
D	decimal point (uses locale)
G	group separator (uses locale)
RN	Roman numeral (input between 1 and 3999)
TH or th	ordinal number suffix
EEEE	exponent for scientific notation

SELECT.. Datatype Formatting functions

- `to_char()`

to_char() function can be used to format date or numeric values and output will be treated as string.

Examples:

```
to_char(timestamp '2002-04-20 17:31:12.66', 'HH12:MI:SS')
to_char(now(),  'YYYY-MON-DD')
to_char(125, '999') → 125
to_char(125.8::real, '999D9') → 125.8
to_char(-125.8, '999D99S') → 125.80-
```

SELECT.. Datatype Formatting functions

to_date() Function

```
to_date('05 Dec 2000', 'DD Mon YYYY') → 2000-12-05
to_date('05-Dec-2000', 'DD-MON-YYYY')  -> 2000-12-05
```

to_number() Function

```
to_number('122345', '99G999D9S')  ->  12,234.5
```

SELECT.. Aggregate functions

```
sum()
avg()
count()
min()
max()
string_agg()
```

SELECT.. Joins

Table joining technique allows to combine rows from from 2 or more table. Following join types can be used in PostgreSQL

- Cross Join
- Inner Join
- Natural Join
- Left Outer Join
- Right Outer Join
- Full Outer Join

- Tuning Tips for Joins
  - Always join the tables using JOIN & ON <condition> clause. Do not join using FROM clause
  - Join's use WORK_MEM. Join large tables require more than default WORK_MEM (4MB)
  - Use proper indexing (columns used ON clause). Also consider Materialized Views
  - Upto date optimizer statistics

SELECT Statements.. Joins

Join Methods You Might See in Execution Plans

Join Method	When Used	Notes
Nested Loop	Small tables or with index	Fast if inner table has index
Hash Join	Large tables, no index	Memory-heavy
Merge Join	Sorted tables or indexed	Efficient for sorted data
Materialize	Subquery or CTE used in join	Materializes result before join
Index Scan	With nested loop	Often used together

SELECT Statements.. Subqueries

- Single valued (Scalar) Subquery
- Multi valued subquery (use IN operator)
- Correlated Sub Query
- Performance Tips for Sub Queries
  - Usually scalar subqueries works well as long as proper indexes are created on columns with WHERE & GROUP BY clauses
  - Prefer Joins Over Correlated Scalar Subqueries in SELECT (Optimizer does this automatically wherever possible)
  - Ensure Indexes on JOIN/Filter Columns Used Inside Subquery
  - Prefer EXISTS Over IN with Subqueries (Especially Large Ones)
  - Avoid NOT IN with Nullable Columns instead use NOT EXSITS
  - Correlated subqueries can be replaced with joins/ctes

Advance Datatypes

Advanced Datatypes.. Array Columns

- An array can store multiple values of same type.
- GIN index is useful for speeding up array searches
- A B-Tree index can only be used if full Array comparison needs to done
- Create a table with Array column

```
CREATE TABLE <tablename> ( <normalcolumn>   <datatype>,  <arraycolumn_name>
<datatype>[]  ...  );
```

- Example

```
create table arr_test (custid int, custname varchar(25),  Address varchar(50)
phones  int[] );
.
```

Advanced Datatypes.. Array Columns

- An array can store multiple values of same type.
- GIN index is useful for speeding up array searches
- A B-Tree index can only be used if full Array comparison needs to done
- Create a table with Array column

```
CREATE TABLE <tablename> ( <normalcolumn>   <datatype>,  <arraycolumn_name>
<datatype>[]  ...  );
```

- Example

```
create table arr_test (custid int, custname varchar(25),  Address varchar(50)
phones  int[] );
.
```

Advanced Datatypes.. Array Columns

Insert can be done along with other columns

```
Syntax:
INSERT INTO  <tablename> VALUES (v1, '{pray elements separated by commas}' );
Example
INSERT INTO arr_test values (10, '{1000,2000,3000}');


select * from arr_test;
select arr_col[2] from arr_test;
select * from arr_test where arr_col[2] = 2000;


UPDATE arr_test SET arr_col = '{25000,25000,27000,27000}' WHERE id=10;
UPDATE sal_emp SET arr_col[2] = 5000 WHERE id=10;
```

Advanced Datatypes.. Composite Columns

- Needs to be created before user
- It is a RECORD type data structure consisting of columns belonging to existing datatypes
- A B-Tree index created on full columns but it is not efficient
- B-Tree indexes can be created on individual fields
- Define a Composite Datatype
- syntax

```
CREATE TYPE <composite type name> AS (
    <col1>            <datatype>,
     <col2>            <datatype>,
);
```

- Example:

```
CREATE TYPE inventory_item AS (
    name            text,
    supplier_id     integer,
    price           numeric
);

.
```

Advanced Datatypes.. Composite Columns

- Lets say the table is as below

```
create table  weekly_stock (
  Weekofyear  int,
  item  inventory_item,
  year  decimal(4)   );
```

INSERT

```
Insert into  weekly_stock values  (12, '("Coke", 1001, 134.00)', 2020);
```

SELECT

```
select a particular column from composite type need the notation of  (<composite type name>).<columnname>
Select * from weekly_stock;
Select (item).name from weekly_stock where item.supplier_id = 1001;
() must be used to enclose composite type columns when accessing
```

UPDATE

```
update weekly_stock set item.name='PEPSI' where (item).supplier_id = 1001;
```

Advanced Datatypes.. HSTORE Columns

- Hstore columns allows us to store data as a set of unique identifiers each of which have an associated value.
- Each of the identifier will have a value. Basically I it KEY → Value pair
- HSTORE datatype is comparable to TEXT datatype.
- An extension needs be installed in the database in which HSTORE columns are needed
- EXTENSION

```
create extension hstore;
```

- Create table

```
create table  customers ( cust_id  int,  cname  varchar(30), address  hstore);
```

- INSERT

```
insert into customers  values (10, 'Scott', ' "Hno" ⇒ "134", "screet ⇒ JamesStreet", "locality ⇒
"Jubileehills", "city" ⇒ 'Hyderabad“’);
```

- SELECT

.

Advanced Datatypes.. HSTORE Columns

- SELECT

```
Select * from customers;
select name,  address → 'city'  city from customers;
select  cname,  address →  city  city from customers where  address → 'locality' = 'JamesStreet’;
```

- UPDATE

```
update  customers set address = address ||'"state"=→"Telangana"'::hstore where cust_id=10;
```

.

Advanced Datatypes.. JSON/JSONB Columns

- PostgreSQL provides JSON & JSONB datatypes.
- Inputs are validated and rejected if they are not confirming to standard JSON syntax

JSON	JSONB
Stores data in plain text	Stores data in binary format
Preserve the indentation of input data	Does not preserve indentation of input data
Does not support indexing	Supports Indexing (GIN indexes)
Does not allow adding or removing field	Allows adding OR removing fields

- JSON Objects
  - JSON Document

    { “key”:”value”, “key2”:<values>, … }
  - JSON ARRAY

    [v1, v2, v3, …]
  - Complex JSON objects

    {“key”:”values”, “key2”: [v1,v2..], “key3”: {“key_a”:”value”, “key_b”:”value”,..}}

Advanced Datatypes.. JSON/JSONB Columns

Operator	Right Operand Type	Description	Example	Example Result
->	int	Get JSON array element (indexed from zero, negative integers count from the end)	'[{"a":"foo"},{"b":"bar"},{"c":"baz"}]'::json->2	{"c":"baz"}
->	text	Get JSON object field by key	'{"a": {"b":"foo"}}'::json->'a'	{"b":"foo"}
->>	int	Get JSON array element as text	'[1,2,3]'::json->>2	3
->>	text	Get JSON object field as text	'{"a":1,"b":2}'::json->>'b'	2
#>	text[]	Get JSON object at specified path	'{"a": {"b":{"c": "foo"}}}'::json#>'{a,b}'	{"c": "foo"}
#>>	text[]	Get JSON object at specified path as text	'{"a":[1,2,3],"b":[4,5,6]}'::json#>>'{a,2}'	3

Advanced Datatypes.. JSON/JSONB Columns

Operator	Right Operand Type	Description	Example
@>	jsonb	Does the left JSON value contain the right JSON path/value entries at the top level?	'{"a":1, "b":2}'::jsonb @> '{"b":2}'::jsonb
<@	jsonb	Are the left JSON path/value entries contained at the top level within the right JSON value?	'{"b":2}'::jsonb <@ '{"a":1, "b":2}'::jsonb
?	text	Does the string exist as a top-level key within the JSON value?	'{"a":1, "b":2}'::jsonb ? 'b'
?|	text[]	Do any of these array strings exist as top-level keys?	'{"a":1, "b":2, "c":3}'::jsonb ?| array['b', 'c']
?&	text[]	Do all of these array strings exist as top-level keys?	'["a", "b"]'::jsonb ?& array['a', 'b']
||	jsonb	Concatenate two jsonb values into a new jsonb value	'["a", "b"]'::jsonb || '["c", "d"]'::jsonb
-	text	Delete key/value pair or string element from left operand. Key/value pairs are matched based on their key value.	'{"a": "b"}'::jsonb - 'a'
-	integer	Delete the array element with specified index (Negative integers count from the end). Throws an error if top level container is not an array.	'["a", "b"]'::jsonb - 1
#-	text[]	Delete the field or element with specified path (for JSON arrays, negative integers count from the end)	'["a", {"b":1}]'::jsonb #- '{1,b}'

Advanced Datatypes.. JSON/JSONB Columns

- Creating a Table With JSONB

```
create table emp_json (id serial, details JSONB);
```

- INSERT

```
insert into emp_json (details) values ('{ <valid JSON document> }'), ('{ <valid JSON document> }'), ..
Insert into emp_json (details) values (‘{ename:”John”, job: “Analyst”, salary:35000 }’);
```

- SELECT

```
select * from emp_json;
select details → 'ename' from test;
select details ⟶ 'ename' from test;
select details ⟶ 'ename', details ⟶ 'job', details ⟶ 'sal' from test where details ⟶ 'ename' = 'FORD';
select details ⟶ 'ename' as ename, details ⟶ 'job' as Job, details ⟶ 'sal' as sal from test where cast
(details ⟶ 'sal' as integer) ≥ 2000;
select details ⟶ 'ename' as ename, details ⟶ 'job' as Job, details ⟶ 'sal' as sal from test order by
cast (details ⟶ 'sal' as numeric);
select details ⟶ 'job' as Job, count(*) from test group by details ⟶ 'job'
```

Advanced Datatypes.. JSON/JSONB Columns

- Check if the JSON Field Exists

```
select details ->> 'ename' as ename, details ->> 'job' as Job, details ->> 'sal' as sal from test where details ? 'sal';
```

- Check if a Key Value Pair Exists in

```
select details ->> 'ename' as ename, details ->> 'job' as Job, details ->> 'sal' as sal from emp_json where details @> '{"ename":"KING"}';
```

- Updating a Field (Only JSONB)

```
update emp_json set details = jsonb_set(details, '{sal}', '1000',false) where details ->> 'ename' = 'SMITH';
```

- Removing a Field (Only JSONB)

```
update emp_json set details = details - 'comm' where details ->> 'ename' = 'SMITH';
```

Advance Querying

Advanced Querying.. CTE common table expression

- Common Table Expressions (CTE) are useful to simplify the complex queries
- Result set of CTEs auxiliary query is temporary until primary query execution completes.
- Materializes the CTE auxiliary query results thus giving the better performance and reduces complexity
- Syntax

```
WITH <cte_name> AS (auxiliary query) primary_query;
```

<cte_name> : a temporary name given to the resultset of aux query
Auxiliary_query : A valid query which will be executed before primary query
Primary_query : Actual query. This query can refer results of auxiliary query by <cte_name>

- Example

```
WITH emp_salgrade AS (select ename, job, sal, deptno, grade from emp join salgrade on sal between
losal and hisal) select ename, job, sal, grade from emp_salgrade es join dept d on
es.deptno=d.deptno;
```

- Example with Data Modifying Statement

```
WITH del_table AS (delete from visitor_logs where log_date < CURRENT_DATE - interval '1 month'
RETURNING *) insert into visitor_archive select * from del_table;
```

Advanced Querying.. Window Functions

- min(),
- max(),
- avg()
- rank() – ranks the output
- row_number() - generates the row numbers
- lag() - Showing the value of previous row in current row
- lead() - Showing the value of next row in current row
- last_value() - Showing last value
- first_value() - Showing first value

Advanced Querying.. Window Functions Examples

- min()

```
select ename, job, sal, min(sal) over (partition by job) from emp;
select ename, job, sal, min(sal) over (partition by job order by sal desc ) from emp
select ename, job, sal, min(sal) over () from emp ;
```

- rank()

```
select ename, job, sal, rank() over (order by sal desc ) from emp;
select ename, job, sal, rank() over (partition by job order by sal desc ) from emp;
```

- row_number()

```
select ename, job, sal, row_number() over (order by sal desc ) from emp;
```

- lag()

```
select ename, job, sal, lag(sal,1) over (partition by job order by sal desc ) as prev_sal from emp;
```

- lead()

```
select ename, job, sal, lead(sal,1) over (partition by job order by sal desc ) as next_sal from emp;
```

- last_value()

```
select ename, job, sal, last_value(sal) over (partition by job order by sal desc ) from emp;
```

- first_value()

```
select ename, job, sal, first_value(sal) over (partition by job order by sal desc ) from emp;
```

Advanced Querying.. Full Text Search

- **Document :** Document is a search unit and it is a text body like a magazine article, email etc.. Usually a TEXT column from a table.
- **tsvector:** A data type that stores preprocessed documents in a format optimized for searching.
- **tsquery:** A data type used to represent search queries.
- **@@ (match operator) :** This operator is used to match the Document (tsvector) with search query (tsquery). Returns true or false depending if the match is found or not.
- Text Analysis
  - **Tokenization**: Breaking down text into tokens (words).
  - **Normalization:** Converting tokens into lexemes (normalized words) to handle different forms of the same word.
  - **Stop Words:** Common words that are ignored during searches (e.g., "the", "and").

Advanced Querying.. Full Text Search Query Operators

Operator	Description	tsquery Example
&	AND operator	‘cat & fat’
|	OR Operator	‘cat | rat’
!	NOT operator	‘!rat’
<->	FOLLOWED BY	‘fat <-> cat’
()	Parenthesis to group search terms	‘(fat <-> cat) & hat’

Advanced Querying.. Full Text Search Functions

Function	Description	Example
to_tsvector(<config>, <doc>)	Converts a text into searcheable vector	Select to_tsvector(english, ‘sam the fat cat‘);
To_tsquery(<config>, <search query>	Converts words to tsquery	Select to_tsquery(english, ‘sam & fat‘)
tsvector_to_array (tsvector)	Converts tsvector to an array consisting of lexemes	Tavector_to_array(comments)
array_to_tsvector (text[])	Converts an array to tsvector	
plainto_tsquery ([config regconfig,] query text)	Converts plain text to tsquery	
phraseto_tsquery ([config regconfig,] query text)	Converts text to tsquery ignoring punctuations	

Advanced Querying.. Full Text Search Configuration

- **default_text_search_config**: This parameter specifies the default text search configuration to be used by text search functions if an explicit configuration is not provided. It can be set in the postgresql.conf file or for an individual session using the SET command.
- **dictionaries:** PostgreSQL supports various dictionaries like simple, synonym, thesaurus, ispell, and snowball. These dictionaries help in processing and normalizing text during searches.
- **parsers:** The text search parser is responsible for breaking down text into tokens. You can specify a parser when creating a text search configuration.
- **stop words:** These are common words that are usually ignored during searches. You can define stop words in your dictionaries to improve search efficiency

Advanced Querying.. Full Text Search Examples

- Example1:

```
select 'Sam the fat cat with a hat can see the Jam'::tsvector @@ 'cat & hat'::tsquery;
 ?column?
----------
 t
```

- Example2

```
select 'cat & hat' @@ 'Sam the fat cat with a hat can see the Jam'::tsvector;
 ?column?
----------
 t
(1 row)
```

- Example3

```
select 'Sam the fat cat with a hat can see the Jam'::tsvector @@ 'cat & rat'::tsquery
;
 ?column?
----------
 f
(1 row)
```

Above match failed because there is no ‘rat’ in document.

Advanced Querying.. Full Text Search Examples

- Example4

```
select 'Sam the fat cat with a hat can see the Jam'::tsvector @@ '!rat'::tsquery;
 ?column?
----------
 t
(1 row)
```

- Example5

```
select 'cat ran the running test'::tsvector @@ 'run'::tsquery;
 ?column?
----------
 f
(1 row)
```

- Example6

```
select to_tsvector('cat ran the running test')  @@ to_tsquery('run');
 ?column?
----------
t
(1 row)
```

Advanced Querying.. Full Text Search Examples

- Creating a table with tsvector Column

To perform FT Search on table columns, it is recommended to have a tsvector column in table rather using **to_tsvector()** function for every search. This also allows to create a index for FTS.

- Example:

```
CREATE TABLE user_feedback (id INT PRIMARY KEY, fb_date TIMESTAMP, rating INT CHECK (rating between 1 and 5), comments  TEXT,  ft_search  TSVECTOR GENERATED ALWAYS to_tsvector(comments) stored);
```

- Indexes of tsvector Columns

  GIN indexes are preferred on tsvector column however GiST indexes also can be used.

Views & Temp tables

Advanced Querying.. Normal View

- Before a view is used it needs to be created.
- syntax

```
CREATE [ OR REPLACE ] [ TEMP | TEMPORARY ] VIEW name
    AS query
    [ WITH [ CASCADED | LOCAL ] CHECK OPTION ]
```

- Altering the View

```
ALTER VIEW [ IF EXISTS ] name OWNER TO { new_owner | CURRENT_USER | SESSION_USER };
ALTER VIEW [ IF EXISTS ] name RENAME TO new_name;
ALTER VIEW [ IF EXISTS ] name SET SCHEMA new_schema;
ALTER VIEW [ IF EXISTS ] name SET ( view_option_name [= view_option_value] [, ... ] );
ALTER VIEW [ IF EXISTS ] name RESET ( view_option_name [, ... ] );
```

- Dropping the View

```
DROP VIEW  name [CASCADE];
```

CASCADE option deletes all dependent objects as well

Advanced Querying.. Materiliazed View

- Materialized Views saves the data offline to source tables
- They are good in improving performance
- Materialized view can be updated with latest data by using REFRESH MATERIALIZED VIEW command
- Indexes can be created on materialized views
- syntax

```
CREATE MATERIALIZED VIEW name AS query;
```

- Refreshing Materialized View

```
REFRESH MATERIALIZED VIEW view name;
```

Advanced Querying.. TEMP Tables

- PostgreSQL allows a session to create a temporary table which is visible for this session
- Temporary table are dropped as soon as session logsoff
- syntax

```
CREATE TEMPORARY TABLE temp_table_name(  … ) [ON COMMIT DELETE ROWS | DROP];
```

Advanced Querying.. Table Inheritance

PostgreSQL supports table inheritance , which allows you to create a hierarchy of tables where a child table inherits the structure (columns and constraints) from a parent table .

Create Parent Table:

```
CREATE TABLE customers (custid serial PRIMARY KEY, cname text NOT NULL, address text);
```

Create Child Table:

```
CREATE TABLE customer_child (  ) INHERITS (customers)
```